

Feasibility Pump with Objective Cutoff: A Computational Study

Matteo Fischetti*

September 2026

Abstract

A common way to make the Feasibility Pump (FP) objective-aware adds an objective function cutoff to the model and runs the pump as a black box. In their original paper, Fischetti, Glover and Lodi (FGL) warned against this: rounded points feasible for the original constraints but not for the cutoff are lost unless one looks at them, and prescribed a recovery, a feasibility test or an LP with the integers fixed and no cutoff. Commonly used implementations do not follow the prescription. SCIP's, for one, uses no cutoff at all and weights the objective instead, so that a user who wants a cutoff adds it to the model and gets no recovery. We measure what both omissions cost. On a from-scratch pump, on covering problems two thirds of the feasible rounded points fall above the cutoff, and the lost improving points pile up just above it at three levels. On MIPLIB 2017 we run the pump standalone inside SCIP, crossing three treatments of the rounded point, none, test or completion, with several cutoff levels, and revalidate every solution. Adding FGL's cutoff and recovery to SCIP's objective-weighted pump beats it, in a comparison added after the pre-declared design, on 116 of the 176 instances (out of 424) where it reaches a solution, loses on 9, and lowers the median final gap from 59% to 30%. The paper thus quantifies, on public instances and with a public artefact, what the black-box use of the cutoff loses and what each recovery action is worth.

Keywords Feasibility pump; Primal heuristics; Mixed-integer programming; Objective cutoff; MIPLIB 2017; SCIP

Mathematics Subject Classification 90C11; 90C59; 90C10

1 Introduction

Finding a first feasible solution of a Mixed-Integer Linear Program (MILP) of practical interest is often a hard task in its own right, and one on which the rest of a branch-and-cut run depends. The Feasibility Pump (FP) of Fischetti, Glover and Lodi [12], FGL in what follows, looks for a feasible solution of the program $\min\{c^\top x : x \in P, x_j \in \mathbb{Z} (j \in I)\}$, $P = \{x : Ax \geq b, l \leq x \leq u\}$. Let $B = \{j \in I : l_j = 0, u_j = 1\}$ be the set of binary variables, which we assume nonempty. The pump maintains two sequences of points that satisfy complementary halves of the problem. The LP iterates $\tilde{x}_0, \tilde{x}_1, \dots$ live in P and are in general fractional; the rounded points $\hat{x}_0, \hat{x}_1, \dots$ are integral on I by construction and in general violate $Ax \geq b$. Starting from the LP optimum $\tilde{x}_0$, each round takes the current LP point, rounds it to $\hat{x}_k$ and projects that back onto P ,

$$\tilde{x}_{k+1} = \arg \min\{\Delta(x, \hat{x}_k) : x \in P\}, \quad \Delta(x, \hat{x}) = \sum_{j \in B: \hat{x}_j=0} x_j + \sum_{j \in B: \hat{x}_j=1} (1 - x_j), \quad (1)$$

*Department of Information Engineering, University of Padova, via Gradenigo 6/A, 35100 Padova, Italy.
matteo.fischetti@unipd.it

which is linear as written. Distance $\Delta(\cdot, \cdot)$ is taken on the binary variables, as in [12], and general integer variables are handled by adding to the LP an auxiliary variable that linearizes $|x_j - \hat{x}_j|$, as in [3] (the instances of this paper are mostly binary). The pump stops when $\tilde{x}_{k+1}$ is integral, as that point is then a feasible solution. When the rounding repeats itself, the pump perturbs $\hat{x}_k$ before projecting again: it flips the T components on which $\tilde{x}_k$ and $\hat{x}_k$ disagree most, with T drawn in [10, 30], integral components included when fewer than T are fractional [12]. Throughout, $\hat{x}_k$ denotes the point the pump projects onto P , that is, the perturbed point at the rounds where a perturbation is applied. Note that every mechanism discussed below acts on the LP (1): it changes the polyhedron the LP iterate is projected onto and leaves the rounding alone. The pump has a large progeny: different projections [6, 2], better randomization [11], and a weighted objective term in place of a pure distance [1, 20] (which is what SCIP runs by default). The pump has also been read as an alternating direction method, with a convergence theory and a penalty in place of the random perturbation [15]. The reader is referred to the survey [4] and to the recent book [5] for an in-depth discussion.

The cutoff, and the warning that came with it. The pump as stated does not take the objective into account. Section 3.2 of [12] proposed the simplest remedy: once an incumbent of value z_{inc} exists, add to the LP the so-called *objective cutoff*

$$c^\top x \leq U, \quad U = z_{\text{low}} + \lambda(z_{\text{inc}} - z_{\text{low}}), \quad \lambda \in (0, 1], \quad (2)$$

with z_{low} a lower bound on the optimum that the solver has at hand. Here $\lambda = 1$ puts the cutoff at the incumbent, so that no improving solution is excluded, and a smaller λ pushes it towards the lower bound. As a matter of fact, this is also what a practitioner does when the pump is a black box: add the constraint statically to the model and run the pump on the restricted problem, hoping for a feasible solution of it. The same section, however, warned against doing only that. Because of the cutoff, “it is often the case that the integer components of x [...] define a feasible point for the original system $Ax \geq b$, $l \leq x \leq u$, but not for the current one”. The remedy was to look at the rounded point every round and, on a mixed problem, to “solve the LP $\min\{c^\top x : Ax \geq b, l \leq x \leq u, x_j = \hat{x}_j \forall j \in I\}$ and compare the corresponding solution (if any exists) with the incumbent one”. Cutoff and recovery were thus designed as two parts of one scheme.

Later work replaced the cutoff by a weighted objective [1], mentioned it in passing [14], or dropped the objective altogether [7, 9]. The pump of SCIP [8, 21], the implementation we work in, keeps neither part: it adds no cutoff and never tests the rounded point. To the best of our knowledge, what the black-box use of a cutoff costs, and what the recovery is worth, has not been measured. In the present paper, we measure both.

What we find. In our view, the main contributions of our work are as follows.

- (i) A census, on a from-scratch pump, of the feasible rounded points that fall above the cutoff and are lost. Their share is large, two thirds on covering problems with the pump’s perturbed roundings counted in, and the lost improving points are not spread over the band between the cutoff and the incumbent but pile up just above the cutoff, at each of the three levels tested (Section 2).
- (ii) The restoration of FGL’s scheme inside SCIP, which we regard as our main result. The *completion* of [12] is the LP that fixes the integer variables at the rounded point and re-optimizes the continuous ones over the original constraints, the cutoff removed (Section 3, where the cheaper *test* of the rounded point is also defined). The objective-weighted pump of [1], as SCIP runs it, with the cutoff at mid-gap and the completion added to it, beats that pump alone: it wins on 116 instances of 176, ties on 51 and loses on 9, and

lowers the median final gap from 59% to 30%. Against the black-box cutoff alone it wins 87–14 (throughout, a – b counts the instances on which the first variant is better and those on which it is worse, paired per instance, ties left out), and against the cheapest of the recovering variants, the test of the rounded point without a cutoff, 114–10 (Section 6). The black-box cutoff already helps, 90–14 over SCIP’s own pump (the *plain pump* of Section 5); the recovery is what makes it help more, and the completion pays where the cutoff is tight, on mixed problems, and not detectably elsewhere.

- (iii) The limits of the scheme. Where the plain pump finds nothing, no variant shows a detectable advantage, and a cheap fallback that anchors the cutoff to the rounded point before an incumbent exists is harmful everywhere. Finally, asked for a solution below a stated target, with the FGL pump written from scratch and no objective weighting, the cutoff in the model, with the rounded point checked against that model, is a strong baseline: it is beaten only at a shallow target, almost only on mixed problems, only by the completion, and not significantly once the declared family of tests is corrected for (Section 7).

A simple geometric phenomenon lies behind these results. The rounded point $\hat{x}$ is often feasible for the original constraints—on covering problems almost always. When it is, it typically costs more than the LP iterate, which typically sits on the hyperplane $c^\top x = U$ when the cutoff bites. Hence, the rounded point violates (2), lies outside the polyhedron the next LP projects onto, and a pump that does not look at it never sees it again. In other words, the loss is a matter of support rather than of weights: a weighted objective term discourages such a point, whereas a cutoff removes it from the polyhedron.

The paper is organized as follows. Section 2 presents the phenomenon on a from-scratch pump. Section 3 states the recovery at its two levels and the design principle that makes an invalid constraint in the projection LP legitimate, namely, that acceptance is decided by a second LP that carries the model’s constraints and none of the invalid ones. Section 4 reads SCIP’s implementation and lists what we added to it. Section 5 describes the two-factor study, ten cells of a fifteen-cell grid, run inside SCIP on all the MIPLIB 2017 instances with mostly binary variables, with the same instances, seeds and budget in every cell, and with every solution revalidated on the original model. Sections 6 and 7 report, respectively, where the recovery helps and where it does not. Finally, conclusions are drawn in Section 8, together with what the setting does not show.

2 The phenomenon: feasible rounded points above the cutoff

We begin by describing the phenomenon on a from-scratch pump. With the cutoff in place the LP minimizes a distance to $\hat{x}$ subject to $c^\top x \leq U$. If the projection without the cutoff has $c^\top x > U$ at its optimum, then by convexity every optimum with the cutoff satisfies $c^\top x = U$, i.e., the LP iterate sits on the hyperplane. (Note that this fails at the first round, where $c^\top \tilde{x}_0 = z_{\text{LP}} \leq U$, with z_{LP} the value of the first LP, and the constraint is inactive.) In what follows, let $\delta = c^\top \hat{x} - c^\top \tilde{x}$ denote the amount by which rounding worsens the cost, the *rounding gap*. When $\delta > 0$ the two sequences are separated by the cutoff, $\tilde{x}$ on the hyperplane and $\hat{x}$ strictly above it. For the rounding to be able to land below the incumbent, one needs

$$\delta \leq z_{\text{inc}} - U = (1 - \lambda)(z_{\text{inc}} - z_{\text{low}}) : \quad (3)$$

the rounding gap must not exceed the room the cutoff leaves. This is a heuristic criterion, since lowering U moves $\tilde{x}$ and hence δ , but it separates problem classes before anything is run. On set covering the median of $\delta/(z_{\text{inc}} - z_{\text{LP}})$ is +0.62, because with nonnegative costs rounding up a fractional cover is feasible and never cheaper. As a consequence, (3) fails at $\lambda = 0.5$.

On the MIPLIB instances of the same sample the median is close to zero, and the criterion holds for every cutoff short of the incumbent (in the target experiment of Section 7, on 134 MIPLIB instances, rounding lowers the cost in about nine rounds out of ten). These and the following measurements come from a from-scratch pump with cutoff (Table 2 in Section 5 places this census beside the two other experiments of the paper), written on top of Gurobi’s LP solver (version 13.0.3), on 37 pure-binary instances (16 set covering from OR-Library, 11 from MIPLIB 2003, 10 from MIPLIB 2017). The runs use $z_{\text{low}} = z_{\text{LP}}$ and hold z_{inc} fixed at a reference value, namely, the first solution that the same pump finds without a cutoff on the same seed, so that the census is taken against a cutoff that does not move. The 37 are what a stratified sample leaves: 20 pure-binary instances per family, drawn with a fixed seed across three size classes (below 1,000, below 10,000 and above 10,000 variables), two of them listed in both MIPLIB releases, hence 58 distinct instances; on 21 there is no band to survey, because the pump without a cutoff finds no solution within the limit (14) or its first solution already equals z_{LP} (7). There are three seeds per instance and level, each run on one thread of the Xeon E3-1220 v2 blades of Section 5 and stopped at 2,000 rounds or 120 seconds; the measurements are reported in full in the accompanying material. Figure 1 shows the two sequences of that pump on one set covering instance over its first 150 rounds with the cutoff, at $\lambda = 0.5$. In the figure, filled circles are feasible improving points above U , which the pump discards; triangles are feasible points at or below U ; hollow circles are feasible points at or above the incumbent; crosses are infeasible roundings; at the rounds where a perturbation or a restart is applied the marker shows the perturbed point, and values above 1.25 are drawn at the top edge. From round 10 to round 106 the pump runs a cycle of length two. The LP iterate sits on the cutoff and is nearly integral, with one to three fractional components; its rounding is feasible and lands just above U (the filled circles on the zero line). The next projection returns the same iterate, the rounding repeats, and the perturbation flips 10 to 30 components: on a covering problem a component flipped to one adds a column and keeps feasibility, so the perturbed point is feasible too, and it lands high in the band (the spikes). A pump that does not look at the rounding discards every one of these points. From round 107 the restart rule of [12] fires every five to seven rounds and flips a fifth of the variables (hollow circles at the top edge, their cost being forty times the band), and every point below U is a plain rounding taken right after a restart. The LP iterate leaves the cutoff at 18 rounds of 150: at rounds 1 and 9, where the projection lands strictly below U , and for one round after each accepted point, where it sits on that integer point.

The figure also shows a lead in time. On this run the first feasible improving rounding appears at round 7, at 0.034 of the band above U , and the first point below U only at round 110: a pump that looks at the rounded point holds an improving solution long before the black-box pump holds any. Over the 37 instances of the census the lead is modest, since on the covering instances the projection itself delivers a point below U within a few rounds: where both events occur, on 28 instances, the first improving rounding comes first on 20, by a factor of 1.4 in rounds (medians over three seeds, then over instances). The lead is decisive where the pump with a cutoff stalls. On `cap6000` the first improving rounding comes at rounds 10 to 27 and the first point below U at rounds 144 to 944, depending on the seed; on `harp2`, `neos-1337307` and `seymour` no seed reaches a point below U within 2,000 rounds or 120 seconds, while an improving rounding appears at rounds 12 to 647. The design of Section 5 does not measure this lead, since its variants start without an incumbent, and we record it here as a property worth measuring on its own.

Lost improving solutions and where to find them. We instrumented that pump so as to test, at every round, whether $\hat{x}$ satisfies the original constraints, cutoff excluded, and if so where it sits relative to U and z_{inc} . Table 1 reports two things. First, the *fraction* of feasible rounded points that lie above the cutoff: a median over all instances between 64 and

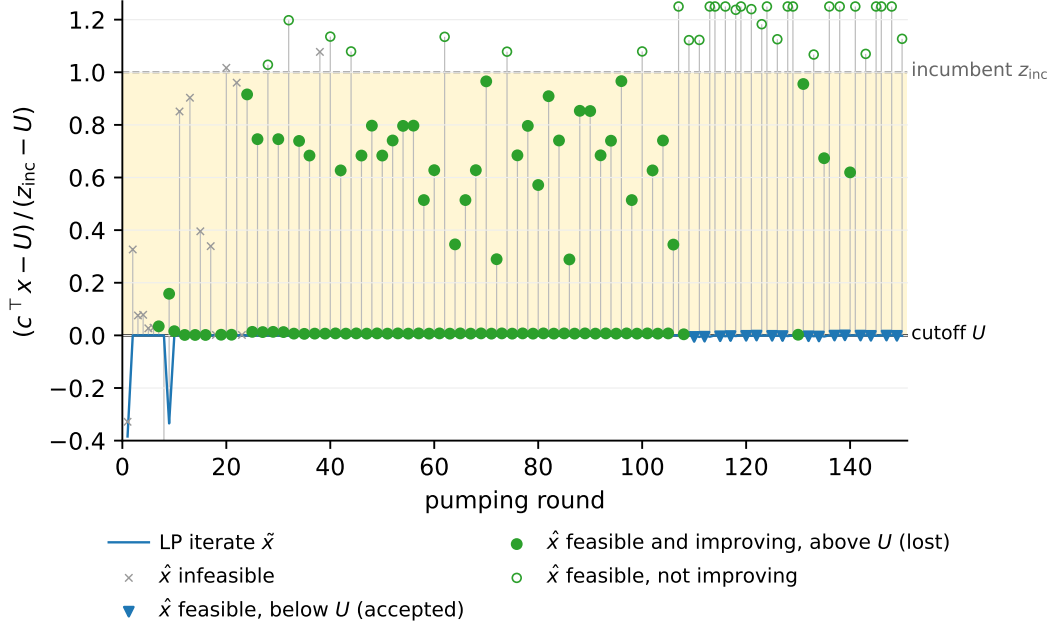


Figure 1: The two sequences of the from-scratch pump with cutoff on `scprh3` (OR-Library, 10,000 columns), seed 0, $\lambda = 0.5$, first 150 rounds. Vertical axis: cost relative to the cutoff U in units of the band $z_{\text{inc}} - U$ (0 on the cutoff, 1 on the reference incumbent, band shaded). The line is the LP iterate $\tilde{x}$; the markers are its rounding $\hat{x}$ at the same round, joined to $\tilde{x}$ by a thin segment (legend in the text). Of 150 roundings, 134 are feasible and 88 are improving and lost, 38 of them perturbed.

66% at the three λ tried, carried by the covering instances, where it is two thirds at every level. On the MIPLIB instances it is much more variable and, except in the smallest cell (five instances at $\lambda = 0.25$), lower, since there the rounding gap is small and the rounded point often lands below the cutoff. Second, the *position* in the band $[U, z_{\text{inc}}]$, 0 meaning “on the cutoff” and 1 “on the incumbent”, of the points that are feasible, above the cutoff and better than the incumbent from which the cutoff was built (that is, the points whose loss is a real loss). Their median position is 0.051 at $\lambda = 0.5$ over the 31 instances that produced such points, aggregated as on the left, with the median taken over three seeds within an instance and then over instances (16,613 points in all). On 29 of the 31 the per-instance median lies in the lower half of the band. The same holds at $\lambda = 0.75$ and 0.25 (30 of 31 and 24 of 29 instances). Of course, the position depends on how the band is defined, and the aggregation script prints every combination. We report the band $[U, z_{\text{inc}}]$ the cutoff actually carves, with z_{inc} the reference incumbent from which U was built, and the improving points only. On seed 0, pooling all feasible rounded points gives 0.23. Taking as the top of the band the incumbent known at that moment gives 0.57, an order of magnitude above the reported value, because that band is much narrower.

Plain and perturbed roundings. The census counts $\hat{x}_k$ as the pump holds it, perturbations included, and Figure 1 shows why this matters: on the covering instances the pump spends a large share of its rounds in the cycle of length two described there, and the perturbed point of every second round is a feasible point above the cutoff. The two kinds separate cleanly in the data, since the distance between $\tilde{x}_k$ and $\hat{x}_k$ equals the fractionality of $\tilde{x}_k$ at a plain rounding and exceeds it by the number of flipped components otherwise. At $\lambda = 0.5$, 40% of the 16,613 lost improving points come from rounds with a perturbation, almost all of them on the

class	fraction above U , median			position in $[U, z_{\text{inc}}]$, median		
	$\lambda=0.25$	$\lambda=0.5$	$\lambda=0.75$	$\lambda=0.25$	$\lambda=0.5$	$\lambda=0.75$
set covering	66.6% (16)	66.9% (16)	66.4% (16)	0.009 (16)	0.015 (16)	0.031 (16)
MIPLIB 2003	53.3% (10)	35.7% (11)	27.1% (11)	0.037 (9)	0.117 (10)	0.151 (9)
MIPLIB 2017	71.4% (5)	17.8% (7)	13.6% (9)	0.058 (4)	0.019 (5)	0.034 (6)
all	66.0%	65.9%	64.4%	0.034	0.051	0.046
instances	31	34	36	29	31	31
median < 0.5	—	—	—	24	29	30

Table 1: Feasible rounded points above the cutoff (from-scratch pump). Left: fraction, medians over three seeds within an instance and then over instances. Right: position in the band $[U, z_{\text{inc}}]$ of the points that are feasible, above the cutoff and better than the incumbent, medians over the seeds available (three on most instances) within an instance and then over instances. The rounded point is counted as the pump holds it, perturbed at the rounds where a perturbation is applied; the text gives the split. In parentheses, the number of instances behind each cell: fractions are computed on the instances with at least one feasible rounded point, positions on those with at least one lost improving point. The last row counts the instances whose median position lies in the lower half of the band.

covering instances. On plain roundings alone the fraction above the cutoff over all instances is 24% instead of 66% (set covering 23% instead of 67%; the MIPLIB rows do not move), and the median position is 0.024 instead of 0.051; at the other two levels the fraction is 26% and 22% and the position 0.009 and 0.041. We keep the full count in Table 1: the perturbed point is the point the pump projects onto P at that round, its loss is as real as any other, and the claim on the position only sharpens when the perturbed points are left out.

Over the three levels tested the pooled fraction lost changes little, while the per-family fractions on the MIPLIB instances do move (Table 1). The normalized position stays between 0.034 and 0.051 at the three levels, and at every level the lost improving points stay in the lower half of the band on most instances (24 of 29 to 30 of 31). If one moves the cutoff down, the room below it shrinks, the band above it widens, and the improving solutions gather again just above the new level (Figure 2). Three levels on three seeds do not establish scale invariance, and we claim none. What they do say is that a policy that only moves the cutoff cannot, by itself, recover what the cutoff excludes.

The loss is a matter of support, not of weights. A natural objection is that a cutoff is a constraint, a constraint has a multiplier, and [1] already replaced the cutoff by a weighted objective term, which reads as follows:

$$\min \frac{1 - \alpha}{\|\Delta\|} \Delta(x, \hat{x}) + \frac{\alpha}{\|c\|} c^\top x, \quad (4)$$

where $\|\cdot\|$ is the Euclidean norm of the coefficient vector of the corresponding linear function. One could argue that, on this view, everything said about the cutoff should have a Lagrangian counterpart; this is, however, not the case, for the following reason. The equivalence between a constraint and a penalty holds for the LP, but the pump is the composition of an LP with a rounding, and the loss above lives on the rounded points—where no duality operates. Under (4) the polyhedron is unchanged, and a costly rounded point that is feasible for the original constraints remains feasible, merely discouraged. By contrast, under (2) the same point lies outside the polyhedron altogether. The two mechanisms therefore differ in which points the projection LP can return: the cutoff changes the polyhedron, the weighted term does not. In

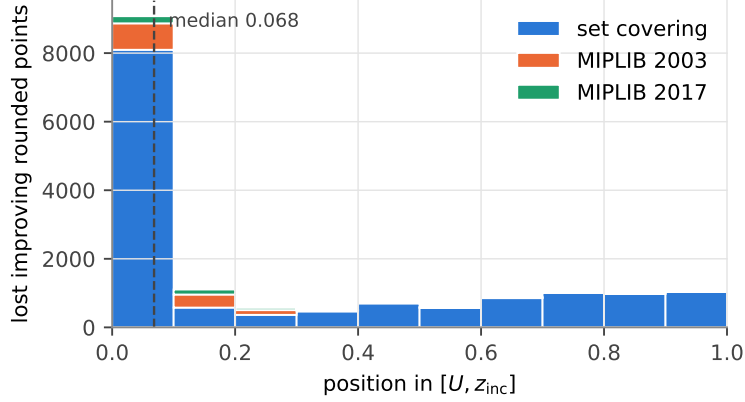


Figure 2: Where the lost improving points sit: position in the band between the cutoff U and the incumbent (0 on the cutoff, 1 on the incumbent), $\lambda = 0.5$, all 16,613 points of the three seeds pooled (the dashed line is their median, 0.068; the median of per-instance medians is 0.051). They pile up against the cutoff.

particular, without a cutoff a feasible rounded point is never lost: if $\hat{x} \in P$ and the round solves the pure projection ($\alpha = 0$ in (4)), the optimal value of (1) is 0 and every optimal solution agrees with $\hat{x}$ on B , so the next LP returns $\hat{x}$ itself on a pure binary problem, or a point of the zero-distance face that differs from it only on the continuous variables on a mixed one. The weighted term does not break this either for small enough α , since below an instance-dependent threshold the optima of (4) are the minimizers of $c^\top x$ on the zero-distance face. Whether SCIP’s schedule, $\alpha = 0.1$ in the first round and divided by ten at every round after it, is below that threshold in the rounds that matter we neither bound nor verify: what happens inside SCIP, where $\alpha > 0$ in every round, is measured in Section 6 and not deduced from the pure-projection limit.

3 Recovery of the rounded point, and why an invalid constraint is legitimate

We next state the recovery at its two levels, and the principle that makes an invalid constraint in the projection LP legitimate. All variants in what follows share the same pump and differ in two things: (i) whether the constraint $c^\top x \leq U$ is added to the projection LP, and at which λ ; and (ii) what is done with the rounded point once it exists.

Two levels of recovery. The cheap level, which we call the *test*, is the one [12] prescribed for pure-integer problems: at every round, test $\hat{x}$ for feasibility against the original constraints, cutoff excluded, and if feasible offer it to the solver. It is one pass over the constraint matrix, hence cheap next to the LP re-solve it may save. It is sound but not sharp on a mixed problem, where $\hat{x}$ carries the continuous components of a projection LP that minimized a distance under a constraint that is not part of the problem. As a completion of the rounded integers those components are arbitrary. As a consequence, the test rejects points for which a feasible completion exists. The sharp level, which we call the *completion*, is the postprocessing of [12]: fix the integers at $\hat{x}$ and minimize $c^\top x$ over the constraints of the model, cutoff excluded. It decides feasibility in that model instead of guessing it, and returns the best completion rather than an incidental one; on a two-stage problem, integer first stage and continuous second, it is the whole question. Observe that “that model” is the one the pump works on, which

inside a solver is the presolved model (Section 5). A completion the presolved model lacks may exist in the original one, so a failed completion is not proof that the fixing admits none. Likewise, the external validation of every accepted solution guarantees soundness, not that nothing recoverable is rejected. We implement the completion as stated, and what is ours is only the placement: the completion runs on a second LP kept apart from the pump’s own, as the two keep separate bases. Between two rounds only the integer bounds of the second LP move, so it reoptimizes with the dual simplex from its own last basis, whereas fixing those bounds inside the pump’s dive would destroy the basis the pump warm-starts from. What this costs in practice is measured, not assumed, in Section 6.

Two decoupled LPs, and what the decoupling buys. The exclusion of the cutoff from the test is not a detail: the projection LP and the LP that decides feasibility are two different linear programs, and keeping them apart is what makes (2) legitimate at all. The first carries a constraint that is *not valid* for the problem, and nothing in the scheme requires it to be the only one. Its role is only to push the LP trajectory towards good solutions, or away from a point the pump keeps returning to. The second sees only the constraints of the model and its objective; it is the certificate, and correctness rests on it alone. Two properties must be kept apart. *Soundness*: no point that violates the model’s constraints is ever accepted, whatever the first LP contains. *Effectiveness*: nothing of the sort is guaranteed, as a wrong bound or a constraint that excludes part of the feasible region can make the first LP infeasible or stop it from producing candidates. What the decoupling buys is therefore the freedom to try such constraints without risking a wrong answer, not a promise that they help; as a by-product, it also gives each of the two LPs its own warm start, from its own last basis, which is better suited to it than the basis of the other. Other invalid constraints have been used in the projection LP: the so-called variable neighborhood pump of [17, 19] adds $\Delta(x, \hat{x}_i) \geq \Delta(\tilde{x}_i, \hat{x}_i)$ permanently, and local branching [13] is the inequality reversed. In this paper the cutoff is the only invalid constraint we use.

The cutoff level, and the continuation rule. U is (2) with λ chosen once, z_{low} read from the solver’s dual bound when the heuristic is entered, and U recomputed at every round so as to follow the incumbent down. When the pump finds a solution it does not stop but updates z_{inc} and continues, as otherwise the cutoff would act at most once. Which λ to choose is not obvious: at $\lambda = 1$ nothing improving is excluded, though the constraint still restricts the projection; at small λ the pressure is strong and by (3) most rounded points are lost. We treat λ as the knob it is and measure a range of levels rather than defend one. A band of width $\beta(z_{\text{inc}} - z_{\text{low}})$ below the incumbent is sometimes a more natural way to say the same thing, but it is in fact the same thing, namely, (2) at $\lambda = 1 - \beta$, and we do not present it as a separate policy. In our harness $z_{\text{low}} = z_{\text{LP}}$ (Section 5), so $U \geq z_{\text{LP}}$, the first LP optimum satisfies the constraint, and the projection LP is never infeasible. The fallback of the next paragraph is the one setting in which it can be.

What happens before the first incumbent. Equation (2) needs an incumbent, and before the first one exists there is no cutoff to impose—precisely on the instances where the pump stalls. The cheap way to have a cutoff from the second round on is to fall back on the cost $c^\top \hat{x}$ of the current rounded point as the upper reference until an incumbent exists. It is tempting; the objection to it is, however, substantive: the rounded point is not a solution, its cost bounds nothing, and since $\hat{x}$ need not lie in P the constraint $c^\top x \leq c^\top \hat{x}$ can make the projection LP infeasible. The design of Section 5 treats this fallback as a factor of its own, never bundled with anything else, so that its effect can be read alone.

4 SCIP’s implementation, and what we added to it

We now turn to SCIP’s implementation, read from the code rather than from the papers. Its pump is the file `heur_feaspump.c`. At the development snapshot we pin (commit `dba4b2a` of 24 August 2026, version string 11.0.0, with SoPlex at commit `f0dbc81` of 8 August 2026) it declares itself “objective feasibility pump 2.0” (its `HEUR_DESC` string; the latest release report, [8], describes version 9.0, and the snapshot is the development version after it). It is indeed the objective-weighted pump of [1], with the round-and-propagate stage of [14] (`usefp20`) and a local-branching sub-MIP after a failed pump (`stage3`) available as options that are off by default and stay off in every run of this paper. It contains exactly one call that offers a solution to the solver (`SCIPtrySol`, line 1271 of the file at that commit), made on the LP iterate in the branch taken when it has no fractional variable. The rounded point, by contrast, is built every round, used for distances and cycle detection, and never tested for feasibility, neither directly nor by the postprocessing LP. SCIP imposes no cutoff on the diving LP either: the solver’s cutoff bound is consulted once (`SCIPgetCutoffbound`, line 795), to skip the heuristic when the LP value already exceeds it, and the objective limit of the diving LP is reset to infinity as soon as the pump changes the objective (`SCIPchgVarObjDive`, line 1104, which resets the LP’s cutoff bound in `scip_lp.c`); the objective thus enters only through α in (4). This implementation thus keeps neither of the two coupled parts of the original design, and a user who adds the cutoff to the model gets the black-box use of Section 1.

In our experiments we add to SCIP’s pump the cutoff (parameter `cutlam`) and its fallback (`cutfallback`), the feasibility test of the rounded point (`tryrounded`) and the postprocessing on a clone LP (`lpfix`), the continuation after a solution (`restartonsol`; we keep the word “restart” for the random restart of [12]), a stop after the call (`stopafter`), a treatment-blind delivery of integral iterates, and the completion counters of Section 6. All of them are off by default, so that every variant is the same binary, differing only in parameter values.

5 Experimental setup

We next describe the experimental setup, starting from the reason why we run the pump as a standalone heuristic. Table 2 places the factorial study of this section beside the census of Section 2 and the target experiment of Section 7, which use a different pump and a different cutoff.

The pump as a standalone heuristic. Inside a full branch-and-cut the pump is called once at the root (Section 8). To measure the pump rather than the node, we run SCIP at the snapshot above with `limits/nodes = 1`, all other primal heuristics off (`heuristics emphasis off`, then the pump switched back on with `freq = 1` and `freqofs = 0`), the pump’s own stopping rules lifted in every cell alike (`maxloops = -1`, `maxsols = -1`, `maxstallloops = 106`, `maxlpiterquot = 1000` and `maxlpiterofs = 107`, so that the time limit alone stops it, and `restartonsol on`), and `stopafter`, so that the clock and the solution belong to the heuristic alone. The seed enters through `randomization/randomseedshift`, the time limit is wall-clock time (SCIP’s default clock), and the feasibility test uses SCIP’s default tolerances; the settings file of one cell, and the script that writes all of them, are in the supplementary material. We leave presolving at its defaults, and re-validate every solution on the original, unpresolved model (paragraph *Instances*). Observe that in this harness no cutting plane is ever separated, without any separation setting being changed: the pump keeps its default `beforecuts`, so that at the root, with `freqofs = 0` and no incumbent, SCIP calls it once right after the first LP and before the first separation round; the pump then uses its whole budget, and `stopafter` ends the solve when it returns. Across the campaign logs the separators record zero calls and zero applied cuts. As a consequence, the polyhedron the pump projects onto is

Table 2: The three experiments of the paper at a glance: which pump, how the cutoff is defined and when it is active, what is measured, on what.

	census (Section 2)	factorial study (Sections 5–6)	target experiment (Section 7)
pump	FGL written from scratch on Gurobi 13.0.3, pure distance, T flips on a repeated rounding, restart when a rounding recurs (<code>fp.py</code>)	SCIP’s objective-weighted pump, standalone, with the additions of Section 4	FGL written from scratch on Gurobi 13.0.3, pure distance, T flips, restart also forced every 100 rounds (<code>fp_target.py</code>)
cutoff	$U = z_{LP} + \lambda(z_{inc} - z_{LP})$, $\lambda \in \{0.25, 0.5, 0.75\}$, z_{inc} the first solution of the no-cutoff run on the same seed; fixed, active from the first round	(2) with $z_{low} = z_{LP}$, $\lambda \in \{0.5, 0.75, 0.9, 0.95\}$, or none; z_{inc} the pump’s own incumbent, so U moves; absent before the first solution (fallback variants excepted)	target $U = z_{best} + a(z_{inc} - z_{best})$, $a \in \{0.5, 0.9\}$, z_{inc} from a reference run; NAIVE puts U in the model, TEST and COMPLETION put $U' = U - w(U - z_{LP})$ in the projection LP only; fixed, active from the first round
recovery	none (a census)	none, test, or completion	test or completion on the original model
endpoint	feasible rounded points above U , and their position in $[U, z_{inc}]$	final gap to z_{LP} (E1); found or not (E2)	a feasible point with $c^\top x \leq U$ within the time limit
instances, seeds	37, 3	424, 5	134 (43 benchmark and 91 others; 20 more for tuning), 5
budget	2,000 rounds or 120 s	$20 t_{LP}$, clamped to $[20, 300]$ s	as in the factorial study, here 20 to 100 s

the plain LP relaxation of [12], and the anchor z_{low} of (2) is the first-LP value z_{LP} , fixed for the whole run.

Why we leave the relaxation unseparated. This is not a neutral detail. Indeed, a cut is valid for the integer hull of the model it was separated from, and once a solver holds a finite primal bound that model is the problem restricted to improving solutions. SCIP’s aggregation separator (`sepa_aggregation.c`) tries the objective function with the cutoff bound as right-hand side as one of its starting rows, and root reduced-cost fixing tightens global bounds with the incumbent. A feasible but non-improving point may then be excluded by a cut or a box. For a paper about which feasible rounded points a cutoff makes invisible, that would be a confound in the instrument, and we remove it by leaving the relaxation unseparated. Anyone embedding these variants in a full solve inherits it: soundness is not at risk, since acceptance is decided by the constraint handlers, but a failed completion then means “no completion in the restricted polyhedron”. Presolve is a milder instance of the same point: its dual reductions preserve an optimum, not the feasible set, so the model the pump works on can lack feasible points of the original; hence our decision to re-check every solution on the original model.

Time limit and reference. We give each instance a time limit of $20\times$ the time of its own first LP, clamped to $[20, 300]$ seconds, so that instances of widely different size get comparable numbers of pump rounds rather than comparable seconds. Of the 424 instances analyzed, 221 sit at the floor of 20 s, 122 get the budget the rule dictates, and 81 sit at the cap. The campaign ran on two sets of four-core blades with hyper-threading disabled, Xeon E5-2623 v3 at 3.00 GHz and Xeon E3-1220 v2 at 3.10 GHz. An instance never straddles the two: we run every variant and every seed of it on one blade, and derive its time limit from a first LP timed on that blade, thus letting the budget normalize itself. On ten first LPs run on both, with

identical iteration counts, the older blades are slower by 1 to 12%, 6% in the median. We execute the 60 runs of an instance, and the pilot run described below that precedes them, one after another on a blade allocated exclusively to them, the variants in a random order within each seed. All the comparisons we report are paired within an instance.

Instances. We consider all 463 instances of MIPLIB 2017 [16] in which at least 80% of all variables are binary (a rule that leaves out the models dominated by continuous variables), and run those that survive the exclusions below through every variant and every seed. We select no instance on solution quality. The remaining variables are continuous or general integer, and the pump handles the latter as SCIP does. We read the value and time of the first LP once per instance by a probe with separation off and a limit of 600 s. Thirty-seven instances do not solve their first LP within it, and we exclude them before the experiment begins. Two more (`supportcase1`, `neos-3135526-osun`) lose part of their runs, 45 and 12 of 60, to an LP numerical failure inside SCIP, and we exclude them from the tables. We refer the reader to the supplementary material for the list of all 39 excluded instances. The testbed therefore consists of 424 instances, 12 variants and 5 seeds, for a complete grid of 25,440 runs. We write every solution a run reports to a file, one per run (its final incumbent), and re-check it by a separate SCIP process on the original model. Of the 10,218 solutions reported by the runs of the grid (and of the 237 more reported by the pilot runs and by the runs of one excluded instance), none is infeasible, and the recomputed objective agrees with the reported one on all but six runs of one variant on two instances. On those six the recomputed value is *better*, because the rounded point carried integer variables at $1 - 10^{-15}$ whose image in the original space is worth slightly less. In no case, therefore, is a reported value better than what its file contains.

The pilot defines two subsets, not a filter. We first run the plain pump, that is, SCIP’s own pump with no cutoff and no recovery (variant `none_` of the next paragraph; under the common settings above it is not the stock pump as shipped, since it carries the continuation, the lifted limits and the treatment-blind delivery), on every instance at half the budget the variants get (floored at ten seconds) and on one seed, a run called the *pilot* in what follows. Its outcome splits the testbed into two subsets: on the 176 instances where the pilot finds a solution (subset E1) the question is how much better each variant does; on the 248 where it finds nothing (subset E2) the question is whether anything is found at all. The two subsets are thus defined a priori, by that single run of the pilot. We fixed the whole design (factors, levels, endpoints and the twelve comparisons of Table 3) before looking at any result of the variants (we added two further comparisons later and report them as such). Note that the pilot gates nothing: every variant runs on every instance, and the subsets only decide which question a result answers. The pilot is the method being challenged, so E1 is exactly the ground on which that method already works. Half the budget was enough for the pilot on those instances, so the subset is not made small by a stingy pilot (the plain pump at full budget finds a solution on some seed on 27 further instances of E2, Table 6). The pilot is a single run at half budget, while we judge every variant on the median of five seeds at full budget. The subsets and the endpoint therefore live at different noise levels, and the two can disagree in either direction on a few instances (Sections 6 and 7).

Variants. The design has two factors, namely, what is done with the rounded point (nothing, the test, or the completion) and where the cutoff sits (absent, or (2) at $\lambda \in \{0.5, 0.75, 0.9, 0.95\}$). We run ten of the fifteen cells of the grid: no recovery at $-$ and 0.5 (`none_`, `none0.5`); the test at all five levels (`check_` to `check0.95`); the postprocessing at $-$, 0.5 and 0.95 (`FGL_`, `FGL0.5`, `FGL0.95`). Note that the last two rows of the grid (`check` and `FGL`) coincide on a problem with

no continuous variables: with nothing left to re-optimize, no completion LP is built and FGL_λ runs the same code as check_λ , so that the two cells can differ only through the clock. Where continuous variables are present the two differ in two ways, as the completion accepts points the test rejects and, when both accept, returns the same integers with the continuous part re-optimized on the model. General integer variables make no difference here, since both levels fix every integer variable at its rounded value; the split that matters is the presence of continuous variables after presolve, and Section 6 reads the results on the two classes separately. Here none is the objective-weighted pump of [1] exactly as SCIP implements it, with none of the additions of Section 4 and under the common settings of the previous paragraphs, and is the literature baseline of this study. The subscript is λ and $-$ means no cutoff constraint at all. The grid is thus partial, and we rest the analysis on the pairwise contrasts declared in Table 3 rather than on a factorial decomposition of the two factors. $\text{none}_{0.5}$ is the cell closest to the black-box use of the cutoff (a user of the stock implementation adding the constraint and nothing else), with two differences that should be kept in mind. First, such a user has an incumbent from outside and the constraint is active from the first round, whereas here no incumbent exists at the start and the constraint becomes active only once the pump has found a solution of its own. Second, that user’s U is fixed and the pump stops at the first solution of the restricted model, whereas here U follows the pump’s own incumbents and the pump goes on. The truly one-shot scheme (external U held fixed) is the one run by the from-scratch census of Section 2. $\text{FGL}_{0.5}$ carries both parts of the FGL scheme, cutoff and completion, embedded in SCIP’s pump (not the pump of [12] itself, which is the from-scratch pump of Section 7). Two further variants add the fallback of Section 3 at $\lambda = 0.5$ ($\text{none}_{0.5}^{\text{fb}}$, $\text{check}_{0.5}^{\text{fb}}$). All twelve are the same binary and keep SCIP’s default weighting (4), which is the same in every cell and is not a treatment. Every variant, $\text{FGL}_{0.5}$ included, is in fact the pump of [1] with the parts of [12] added to it, since the two designs are additive rather than alternatives. Recall that α is 0.1 in the first round and is divided by ten at every round after it, so the weighting fades fast, and that we deduce nothing below from the pure-projection limit of Section 2.

Solution delivery is treatment-blind. The stock pump hands its solution to the solver only after the loop, on the integral iterate that stopped it. With the continuation, the loop does not stop, so a variant with no recovery would report a solution only when the clock ran out. A variant with the test would instead report every integral iterate as it passed, thus confounding the treatment with the bookkeeping. We therefore hand in every integral iterate the moment it appears, at the top of the loop and before any variant-specific code runs, in every cell alike. The median time to the first solution is between 3.99 and 4.61 seconds for all ten variants of the grid in E1.

Seeds, aggregation, metrics. We run five seeds per (instance, variant), the same five values for every variant, so that runs are paired by (instance, seed). The unit of inference is the instance. For each variant, we first take the median over its five seeds; we then pair comparisons per instance on that median (a tie is a difference below 10^{-6}), and use a sign test on the non-tied instances. Note that the median over seeds can be 100 on an instance where some seeds of a variant do find a solution, an effect of the aggregation rather than of the endpoint. We declared twelve comparisons (the rows of Table 3) before looking at the data, and repeat them in the two subsets. We apply Holm’s correction [18] within each subset, thus yielding two families of twelve.¹ Under the more conservative reading of a single family of twenty-four, the corrected p of $\text{FGL}_{0.5}$ against $\text{check}_{0.5}$ in Table 3 becomes 0.013 and that

¹Holm’s procedure controls the family-wise error rate: when the null hypotheses hold, the probability that the family contains even one false rejection is at most 0.05. We therefore read a corrected p below 0.05 as a difference established at that level; a corrected p above it means that no difference is detected at the precision of this design, which is not evidence that there is none.

comparison (A vs. B)	instances			p -value		median diff. $B - A$, non-tied
	A better	tie	A worse	sign test	Holm	
check ₋ vs. none ₋	22	154	0	$< 10^{-4}$	$< 10^{-4}$	+17.1
check _{0.5} vs. none _{0.5}	67	93	16	$< 10^{-4}$	$< 10^{-4}$	+2.8
none _{0.5} vs. none ₋	90	72	14	$< 10^{-4}$	$< 10^{-4}$	+11.0
check _{0.5} vs. check ₋	105	59	12	$< 10^{-4}$	$< 10^{-4}$	+12.7
check _{0.95} vs. check ₋	119	50	7	$< 10^{-4}$	$< 10^{-4}$	+9.8
check _{0.95} vs. check _{0.5}	64	73	39	0.0176	0.0528	+0.7
FGL _{0.5} vs. check _{0.5}	27	142	7	0.0008	0.0033	+1.7
FGL _{0.95} vs. check _{0.95}	18	147	11	0.2649	0.2649	+0.1
FGL ₋ vs. check ₋	12	160	4	0.0768	0.1536	+1.4
FGL _{0.5} vs. check ₋	114	52	10	$< 10^{-4}$	$< 10^{-4}$	+13.3
check _{0.5} ^{fb} vs. check _{0.5}	18	73	85	$< 10^{-4}$	$< 10^{-4}$	-7.6
none _{0.5} ^{fb} vs. none _{0.5}	19	67	90	$< 10^{-4}$	$< 10^{-4}$	-17.7
<i>added after the design (post hoc); not in the Holm family</i>						
FGL _{0.5} vs. none ₋	116	51	9	$< 10^{-4}$	—	+13.1
FGL _{0.5} vs. none _{0.5}	87	75	14	$< 10^{-4}$	—	+3.3

Table 3: E1, the twelve comparisons declared before the data were looked at, paired per instance on the final gap: instances on which the first variant is better, tied, worse; sign test on the non-tied; Holm correction over the twelve; and the median of the paired difference in percentage points of gap over the non-tied instances (positive means the first variant is better). The last two rows are the two comparisons added after the design (post hoc), with their raw sign-test p and outside the Holm family.

of check_{0.95} against check_{0.5} becomes 0.26, and nothing below changes sign. Since the subsets are defined by a run of one of the variants, we also ran the twelve comparisons on the full testbed of 424 instances. It turns out that every sign is the same. With E2’s ties added, the completion against the test at mid-gap is 28–387–9 (better–tied–worse, the three-number form used when the ties matter; Holm $p = 0.010$), loose against mid-gap 67–312–45 ($p = 0.14$), and the two fallback comparisons 21–308–95 and 22–302–100 ($p < 10^{-4}$). We added two further comparisons, FGL_{0.5} against none₋ and against none_{0.5}, after the design as the paper’s headline (Section 6), and label them post hoc. They are the last two rows of Table 3, with their raw sign-test p and outside the Holm family; folded with the twelve declared ones into a single family of fourteen, both keep a corrected p below 10^{-4} , and the corrected p of the twelve are unchanged. We report counts per run where they add information, but only descriptively, since runs on the same instance are not independent. The primary endpoint in E1 is the final gap to the LP bound, namely, $100|z - z_{LP}| / \max\{|z|, |z_{LP}|\}$, charged at 100 to a run that finds no solution, at 100 when z and z_{LP} have opposite signs, and at 0 when both vanish. It is thus bounded by 100: a run without a solution scores 100, no worse than any run with one. The secondary endpoints are the time to the first solution, rounds, LP iterations, and, for the completion variants, the number of completion LPs solved and how many of them produced an accepted solution. In E2 the endpoint is found or not found, reported for each pair of variants as a 2×2 table, since the “neither” cell is where most of the mass is.

6 Computational results: FGL’s scheme restored inside SCIP

Table 3 reports, on the 176 of the 424 instances that form subset E1, the twelve comparisons declared in advance, and Table 5, deferred to Appendix A, the twelve variants one by one. We give the main result first; the rest of the section takes it apart into the factors of the design, and the last paragraph discusses the footprint of the completion.

The main result. The objective-weighted pump of [1] as SCIP runs it, with the cutoff at mid-gap and the completion of [12] added to it, against that pump alone: `FGL0.5` beats `none_` on 116 instances, with 51 ties and 9 losses. In addition, it lowers the median final gap from 59% to 30%, with a median paired difference of 13 percentage points of gap where the two differ. Against the black-box cutoff, `none0.5`, the same scheme wins 87–14 (with 75 ties) with a median difference of 3.3 points over the instances where the two differ, and by at least ten points on 34 of those instances. Against the test without a cutoff (the cheapest of the recovering variants) the same pair wins 114–10. Observe that the first two comparisons are not among the twelve declared in advance (Section 5): they are the two post hoc rows of Table 3, and corrected within a family of fourteen their p stay below 10^{-4} . In what follows we take that result apart into its factors.

The black-box cutoff already helps, and the recovery makes it help more. Adding the cutoff at mid-gap to a pump that ignores its rounded points, `none0.5` against `none_`, improves the final gap on 90 instances and worsens it on 14, and the median gap falls from 59% to 40%. The black-box use of the cutoff is not useless; it is, however, wasteful. Testing the rounded point without a cutoff is never worse on the median final gap in E1, `check_` against `none_` 22–0. With the cutoff at mid-gap the same test is worth much more, 67–16 for `check0.5` against `none0.5`, and the median gap drops from 40% to 31%. This is precisely the interaction that Section 2 predicts. Recall that without a cutoff a feasible rounded point is recovered by the projection anyway in the pure-projection limit, and inside SCIP the counters below say the same. In that limit, therefore, testing it could only save time. The count above, 22–0 with a median gain of 17 percentage points (Table 3), is more than a saving of time explains. It is not an effect of the objective weight: rerun with $\alpha = 0$ in every round, the same pair gives 17–158–1 with a median gain of 17.9 points, the seventeen instances being a subset of the twenty-two (supplementary material), and the gain sits entirely on the pure instances. What the test changes is when the pump delivers, inside the loop, and what the continuation does from there; we do not trace it further. By contrast, with a cutoff it is outside the polyhedron, and of the mechanisms considered here only an explicit look at it, by the test or by the completion, keeps it. The test can also cost something, on 16 instances here, since an accepted solution lowers U at once, thus moving the trajectory. At equal recovery the cutoff is a lever, `check0.5` against `check_` 105–12 and `check0.95` against `check_` 119–7, and the diagnostics show why. Without a cutoff the pump performs about 8,000 rounds for 2,000 LP iterations in all, a quarter of an iteration per round, that is, the LP mostly returns the basis it already holds, and wanders; with a cutoff it performs about 800 rounds for 100,000 iterations, so that each projection is a real re-optimization under the pressure of the constraint.

The LP recovery improves on the test where the cutoff is tight, and not detectably elsewhere. At $\lambda = 0.5$, re-solving the continuous variables on the original constraints instead of accepting the rounded point as it stands, `FGL0.5` against `check0.5`, is better on 27 instances, tied on 142 and worse on 7, Holm-corrected $p = 0.003$. At the loose level the postprocessing adds nothing measurable (`FGL0.95` against `check0.95`, 18–11, $p = 0.26$) and without a cutoff it adds little (`FGL_` against `check_`, 12–4, $p = 0.15$). The counters of Table 5 say why. Without a cutoff almost every completion LP is feasible (99.4% of 8.4 million) and almost none is accepted (242), because the projection recovers the point anyway (Section 2) and the completion has nothing to add. With the cutoff at mid-gap 58% of the completion LPs are infeasible, as the fixed integers are off the polyhedron altogether, but the accepted ones are eighty times as many (19,865 against 242), because they are points the projection would have discarded. The effect also sits where the mechanism of Section 2 predicts. Of the 176 instances, 56 have continuous variables after presolve and 120 have none. On the 120 pure binary instances no completion LP is ever built and the two variants are the same code path, so they return the

same final gap on 119 (on the remaining one the clock stopped two identical runs at different rounds). On the 56 mixed instances the comparison is 27–23–6. The two levels thus divide the work between the classes. On the 120 pure instances the test alone, `check`_{0.5} against `none`_{0.5}, is 56–53–11, and it is the whole recovery. On the 56 mixed instances the same comparison is 11–40–5, a difference not established at this size, whereas `FGL`_{0.5} against `none`_{0.5} is 31–22–3: there the recovery is the completion. (The 19 instances of E1 with general integer variables after presolve tie on every one of them between the two levels, 0–19–0.) The 27 gains have a median of 3.3 percentage points of gap and reach 17; the seven losses, six of them on mixed instances, have a median of 0.3 and none exceeds 4. It is worth noting that the effect is confined: the two variants differ on 34 instances of 176, so the median paired difference over all instances is zero, and the 1.7 points of Table 3 are the median over the 34. Among those the completion is the better one in 27, a proportion of 0.79 with exact 95% interval [0.62, 0.91]. Splitting E1 at its median nonzero count, the comparison is 12–3 on the small half and 15–4 on the large one. Neither split is in the declared family: the mixed/pure split is the prediction of the mechanism, and the size split is exploratory. Both are reported with raw sign-test p (0.0003 on the mixed instances, 0.035 and 0.019 on the two halves) and no correction.

The cutoff level. A single comparison between levels was declared, the loose cutoff against mid-gap: `check`_{0.95} against `check`_{0.5} is 64–73–39, raw sign-test $p = 0.018$ and Holm $p = 0.053$. That is not significant after correction, and we do not read it as a result. The four levels with the test have median gaps of 31%, 30%, 28% and 29% at $\lambda = 0.5, 0.75, 0.9, 0.95$. Note, however, that this ordering is one of marginal medians, not a paired comparison, and no test supports it. We read the levels as interchangeable within the resolution of this design, and therefore recommend none of them.

The completion’s footprint. Of course, the completion is not free from a computational viewpoint, and Table 5 shows its footprint: the three completion variants solve between 5 and 8.5 million completion LPs each over their 880 runs, 6,000 to 10,000 per run. Without a cutoff almost none is useful. With the cutoff at mid-gap 3.1 million of the 5.3 million solved at mid-gap are infeasible outright, because fixing the integers of a point the cutoff has pushed off the polyhedron often leaves no continuous completion at all. Each call is a dual re-solve on a fixed face. We timed the whole completion step (bound update, dual simplex, solution retrieval and check) in a rerun of the three completion variants on E1, five seeds, with a clock that leaves the pump’s statistics unchanged (supplementary material). On the 56 mixed instances the completion takes a median 22% of a run’s wall-clock time without a cutoff (interquartile range 17–31%, maximum 54%), 7% at mid-gap (2–18%, maximum 37%) and 13.5% at the loose level (4–27%, maximum 53%); a single completion LP costs 0.3 to 0.5 ms on average, so the share is driven by the number of rounds, and it is lowest where the completion pays. On the 120 pure instances no clone LP is built and the cost is exactly zero. The completion LP does not check the time limit: in the timed rerun 12 of 2,640 runs overshoot it by more than one second, one completion LP taking 8 s on a 20 s budget. Under the same wall-clock budget the median number of rounds falls by 6 to 13% in all three pairs of Table 5 (7,695 $\rightarrow$ 6,720, 834 $\rightarrow$ 780, 944 $\rightarrow$ 820), while the median time to the first solution moves by less than 0.1 s. The completion also changes what is accepted and when U moves, hence the trajectory, so the fall in rounds mixes overhead with a changed trajectory. It is called every round, and its yield is a function of the cutoff. The counters take every call into account, including the many on rounded points already seen, so their ratios are meaningful and their absolute values are not. Finally, on 23 of the 176 instances all twelve variants return the same final gap: five solved to the LP bound by everyone, eight on which the median seed of every variant is left at gap 100, and ten agreeing on an intermediate value. On those eight, however, the pilot’s single run,

and some seeds of several variants, do find a solution. On 13% of the instances, therefore, the choice of policy is immaterial.

7 Where it does not help, and a target experiment

E1 and E2 are two subsets of the same testbed under one pre-declared design, and the twelve comparisons are repeated in both. We next report where that design pays nothing, and then describe a separate experiment that changes the pump and the endpoint. The latter includes one endpoint on which the completion is ahead (without reaching significance) and one on which the cutoff in the model is hard to beat.

Where the pump finds nothing (subset E2). Subset E2 consists of the 248 instances on which the plain pump at half budget finds nothing, and on it the design has no effect we can detect. The ten variants of the grid find a solution on between 22 and 27 instances on at least one seed, the two fallback variants on 11 and 9 (Table 6, in the appendix). Paired per instance on the same endpoint as in E1 (Table 7, in the appendix), none of the twelve comparisons detects a difference. The largest count of decisive instances in either direction is ten, and after Holm every p -value is 1. On 230 of the 248 instances all twelve variants agree, almost always on finding nothing. Per run (Table 8) the four cells read as follows for the ten variants of the grid: only one of the two finds a solution, from none to a couple of dozen runs; both, between fifty and eighty; *neither*, always above eleven hundred of 1,240. The cutoff moves a few runs in each direction with no consistent sign, and the two recoveries move almost none. An undetected difference on this subset is a statement of precision rather than of absence, since with at most fifteen discordant instances per comparison (eleven in the median) the design can only exclude large effects. For the cutoff without recovery the discordant instances are 9 against 6, whose exact 95% interval on the proportion is $[0.32, 0.84]$; since in a paired design the odds ratio is the ratio of the discordant counts, odds ratios between 0.5 and 5 remain compatible with the data. This is where a first solution is hard, and neither recovery has anything to act on, since both work on what happens *after* a rounded point turns out feasible. The pump does not produce feasible rounded points, thus leaving nothing to keep or complete. The completion variants solve 3.2 million completion LPs each in E2, of which between 28 and 57 yield an accepted solution—one in 60,000 to 115,000. Whether a cutoff can be made to help *find* a solution rather than improve one is a question this design does not answer. What Tables 6 and 8 say is that on the hard half of the testbed no variant has a detectable advantage at this precision.

The fallback hurts. Anchoring the cutoff to $c^\top \hat{x}$ before an incumbent exists (Section 3) is the one intervention that hurts—with and without the test. In E1, $\text{check}_{0.5}^{\text{fb}}$ against $\text{check}_{0.5}$ is 18–73–85 and $\text{none}_{0.5}^{\text{fb}}$ against $\text{none}_{0.5}$ 19–67–90, both $p < 10^{-4}$. In addition, the two fallback variants report no solution on 58 and 71 of the 176 instances, which no other variant does on more than three (Table 5). Per instance, the fallback loses a solution on 55 and 68 instances respectively and gains one on none. In E2 it is the only factor that moves the found-or-not counts. Counting an instance as found when at least one of its five seeds finds a solution, the fallback loses a solution on 16 instances and gains one on 2 to 4. The instance totals of Table 6 fall from 23 to 11 and to 9. It turns out that a cutoff needs a bound it can trust, and that the cost of a rounded point is not one.

A clean pump on Gurobi, and a stated target. Two questions remain open from Section 6. The first is whether the findings depend on SCIP’s pump, which carries the objective weighting of [1] and everything else a solver adds. The second is what happens in the situation

the cutoff was designed for: a user who holds a solution and wants a better one *by a stated amount*. We therefore ran a separate experiment, E3 (Table 2), which answers both with the FGL pump written from scratch on Gurobi 13.0.3: pure distance and no objective term, rounding to the nearest integer, with the perturbations and restart rule of [12] as published ($T = 20$, the number of components flipped when the rounding repeats being uniform in $[T/2, 3T/2]$; a cycle window of three roundings; a restart forced every 100 rounds, flipping every binary j with probability $0.2 + |\tilde{x}_j - \hat{x}_j|$). It uses one thread, the dual simplex, Gurobi’s defaults otherwise, and the plain LP relaxation with no cutting planes (see the supplementary material, `fp_gurobi/`, for code, lists and raw results). A *reference run* provides the starting solution z_{inc} . It is that same from-scratch pump with the completion, on a seed of its own, and it is unrelated to the pilot that defines the subsets of Section 5, which is SCIP’s pump. Let z_{best} denote the best value known to MIPLIB 2017; the target then reads as follows:

$$U = z_{\text{best}} + a(z_{\text{inc}} - z_{\text{best}}), \quad a \in \{0.5, 0.9\}. \quad (5)$$

In words, the target closes half of the gap to the best known value, or one tenth of it. U is fixed for the whole run, no policy knows z_{best} , and every policy stops at the first point that is feasible for the original constraints with $c^\top x \leq U$. The endpoint is whether it gets there within the budget, and when. Three policies, described in what follows, use the same pump. NAIVE adds the constraint $c^\top x \leq U$ to the model before the pump and checks every rounded point against that model, constraint included. It is the cutoff as a user imposes it, with the strongest check a pump can make without looking at the original constraints on their own (in fact, the black box of Section 4 does not even test the rounded point). TEST and COMPLETION keep the model intact and put an *inner* constraint $c^\top x \leq U'$, $U' = U - w(U - z_{\text{LP}})$, in the projection LP only, recovering every rounded point on the original constraints, by the feasibility test and by the LP with the integers fixed, respectively. The margin w is a “rounding moat” meant to catch, below U , the feasible roundings that Section 2 finds just above the constraint. Observe that it is the only tuning knob, and that NAIVE is its $w = 0$ point. Every accepted point is re-checked on the original model with all variables fixed to it, at Gurobi’s absolute tolerances; one success in 2,891 failed that check and is counted as a failure. Of the 463 instances of Section 5, 430 have a best known value, and a reconnaissance with the reference run leaves 154 ($43 + 20 + 91$ below). Specifically, we exclude the instances with general integer variables or with quadratic, SOS or general constraints, which the 0–1 pump does not handle, those on which the reference run finds nothing in 150 s or is already at the best known value, and those whose first LP exceeds 5 s. The split is exogenous: the 43 instances of the MIPLIB 2017 Benchmark Set are the primary test set, 20 instances drawn at random outside it tune w , and the remaining 91 are the secondary test set; the 20 tuning instances take no part in the evaluation, which thus runs on 134 instances. Of the 134, 70 are pure 0–1 and 64 have continuous variables. The time limit is as in Section 5, here between 20 and 100 s, with five seeds (0–4; 10–14 for the tuning), the three policies of a seed running simultaneously, one thread each and hence under shared memory bandwidth, on a Xeon E3-1220 v2 blade of Section 5 allocated to them alone: 4,020 evaluation runs. On the tuning instances a rule we declared beforehand picks, among seven widths from 0.02 to 0.50 and the $w = 0$ point that NAIVE provides, the w with most successful (instance, seed) pairs, ties broken by the median censored time. At $a = 0.5$ it picks $w = 0$ for TEST, every positive width losing pairs to NAIVE, and $w = 0.15$ for COMPLETION (69 of 100 pairs against 67, within the noise of 20 instances). At that target, we evaluate TEST at $w = 0.02$, the best positive width, as an exploratory arm (marked as such in Tables 4 and 9; the NAIVE-versus-TEST comparison at $a = 0.5$ in the declared family thus involves it). At $a = 0.9$ the rule picks $w = 0.05$ for both (79 and 86 pairs against 76).

Table 4 gives the outcome, and Table 9, in the appendix, the paired comparisons on the outcome alone, with the time kept apart. At the deep target, $a = 0.5$, the moat buys nothing. The three policies solve 87, 84 and 85 of the 134 instances, and every comparison on the

Table 4: Outcome of the target experiment on the evaluation set. Every instance is run with five random seeds under each policy. An instance counts as *solved* when at least three of its five seeds reach a feasible point with $c^\top x \leq U$ within the time limit (the outcome is the median over seeds of the success indicator); *pairs* is the number of (instance, seed) pairs that reach such a point, out of the $5N$ available. The target is $U = z_{\text{best}} + a(z_{\text{inc}} - z_{\text{best}})$ and w is the “rounding moat” as a fraction of $U - z_{\text{LP}}$: policy NAIVE imposes a static cutoff constraint in the model, while TEST and COMPLETION impose an inner constraint at $U' = U - w(U - z_{\text{LP}})$ and recover a point for the true target by a direct feasibility test and by FGL’s LP completion, respectively. At $a = 0.5$ the TEST policy is evaluated at $w = 0.02$, an exploratory choice (the tuning rule picked $w = 0$, which is NAIVE). Subsets: benchmark instances (the primary test set) and non-benchmark ones; pure 0–1 instances ($n_{\text{cont}} = 0$) and mixed ones ($n_{\text{cont}} > 0$).

Subset	N	NAIVE		TEST		COMPLETION	
		solved	pairs	solved	pairs	solved	pairs
$a = 0.5$	$(w = 0.02 \text{ for TEST, } w = 0.15 \text{ for COMPLETION})$						
all	134	87	438/670	84	424/670	85	430/670
benchmark	43	28	143/215	26	130/215	28	142/215
non-benchmark	91	59	295/455	58	294/455	57	288/455
pure 0–1	70	56	278/350	52	268/350	50	254/350
mixed	64	31	160/320	32	156/320	35	176/320
$a = 0.9$	$(w = 0.05 \text{ for TEST, } w = 0.05 \text{ for COMPLETION})$						
all	134	104	519/670	103	518/670	112	561/670
benchmark	43	32	162/215	32	163/215	35	179/215
non-benchmark	91	72	357/455	71	355/455	77	382/455
pure 0–1	70	62	313/350	63	315/350	63	314/350
mixed	64	42	206/320	40	203/320	49	247/320

outcome over the two test sets pooled is a tie on more than 115 instances (9–118–7 for NAIVE against COMPLETION). Where two policies both succeed, the one without the extra LP is the faster. At the shallow target, $a = 0.9$, the completion behind a narrow moat never loses to NAIVE on the outcome and wins on 8 instances of 134, seven of them mixed (0–126–8, exact $p = 0.008$ on the pooled set; 0–57–7 on the 64 mixed instances, 0–69–1 on the 70 pure ones). It pays its extra LP per round in time on the pure instances (38–15 for NAIVE on the instances both solve). Once the declared family of twelve tests (two values of a by two test sets by three comparisons) is taken into account, nothing survives Holm: on the primary set the completion’s advantage is 0–40–3, $p = 0.25$, and the pooled count is reported for information. The TEST policy with a moat is never distinguishable from NAIVE, at either target, and at no width tried on the tuning set. We also tried a moat set per instance from the observed rounding gap: after $K = 5$ rounds the inner constraint moves to $U' = U - m$, with m the median of $\delta_k = c^\top \hat{x}_k - c^\top \tilde{x}_k$ over the rounds seen, truncated to $[0, U - z_{\text{LP}}]$ (supplementary material). On this pump the LP iterate sits on the inner constraint and rounding lowers the cost in about nine rounds out of ten, so the median gap is nonpositive in three runs out of four and the adaptive moat is zero; where it is not, it changes nothing detectable: over the 134 instances the adaptive TEST and COMPLETION are 1–127–6 and 7–122–5 against NAIVE at $a = 0.5$, 3–125–6 and 8–123–3 at $a = 0.9$ (sign test $p \geq 0.125$), and indistinguishable from the fixed- w policies. A moat scaled on $|\delta|$ instead is harmful (15 instances lost by TEST at either target, $p \leq 0.002$). With the objective weighting removed and the endpoint changed from “a better solution” to “a solution below U ”, the cutoff in the model is thus a strong baseline. It does what the recovering variants do at the deep target, and it is beaten only at the shallow one, almost only on mixed problems, and only by the completion, which recovers points the

feasibility test cannot see, exactly where Section 6 found it useful. The mechanism of Section 2 is visible in the failures. When a recovering policy fails, in 59 to 93% of the runs, depending on policy and target, the pump has produced no feasible rounded point at all, above or below U . When it has, the best of them sits above the target by a median 20 to 35% of the gap $U - z_{LP}$ at the deep target, beyond the width the rule selected, and by 5 to 9% at the shallow one, about the width selected. Widening the moat w moves U' towards z_{LP} , where the pump stops producing feasible roundings at all, and since the relative gap $(U - z_{LP})/|U|$ spans four orders of magnitude across the 134 instances, no single w fits. What E3 does not change is the recommendation: at the shallow target the recovery never loses on the outcome, at the deep one it is within noise of the baseline, and it is what makes an inner constraint legitimate.

8 Conclusions

In the present paper we have addressed a common practice, namely, adding an objective cutoff to the model and running the Feasibility Pump on it as a black box, and we have measured what the practice costs and what the recovery prescribed by the original FGL paper [12] is worth. The practice is not useless. Indeed, on the subset of MIPLIB 2017 where the pump already reaches a solution it improves the final gap on 90 instances of 176 and worsens it on 14. It is, however, wasteful in a specific way. In the from-scratch sample of Section 2, on covering problems two thirds of the feasible rounded points fall above the cutoff (one quarter when the roundings the pump perturbs to leave a cycle are left out), and a smaller share elsewhere. The lost improving points do not spread over the band they could have occupied but crowd its lower end—at every level tried. A pump that does not look at them never sees them again, while a pump that fails to reach a target finds nothing near it (Section 7). [12] said so and paired the cutoff with a recovery; the pump of SCIP dropped both. Restoring the test of the rounded point is never worse without a cutoff, on the median final gap over those 176 instances (the subset called E1 above, 22–0), and wins 67–16 over the black box at mid-gap. Restoring the completion (the LP with the integers fixed and the cutoff removed) improves on the test on 27 instances against 6 on the 56 mixed instances on which it can act at all (27–7 over all of E1, the seventh loss falling on a pure instance), with a corrected p of 0.003. At a loose cutoff or without one the completion adds nothing detectable, as the geometry predicts. Added to the objective-weighted pump of [1] as SCIP runs it, the two parts of the FGL scheme beat that pump alone, winning on 116 instances of 176 and losing on 9, and lower the median final gap from 59% to 30%. Against the cheapest sound variant of the design, the test without a cutoff, the scheme wins 114–10. On the subset where the plain pump finds nothing, no variant shows a detectable advantage at the precision of the design, and the cheap fallback that anchors the cutoff to the rounded point before an incumbent exists is harmful everywhere. Asked for a solution below a stated target, with the FGL pump written from scratch and no objective weighting, the cutoff placed in the model, with the rounded point checked against that model, proves a strong baseline. The recovering variants match it at a deep target, and at a shallow one the completion beats it on eight instances of 134, seven of them mixed, 0–126–8 on the outcome, without losing one. This last count is the pooled one, reported for information. On the primary test set the same comparison is 0–40–3 with $p = 0.25$, and nothing in the declared family of twelve tests survives Holm’s correction, so that the shallow-target advantage of the completion is a lead and not an established result.

A lesson learned is that the cutoff and the recovery that goes with it should be kept together. Our recommendation is therefore not a new heuristic but an old one, restored. For a pump such as SCIP’s, which weights the objective and imposes no cutoff, it means adding the cutoff once a solution exists, together with the recovery that goes with it. For the record, we also tried the cutoff as the pump’s perturbation: a constraint on $c^\top x$ that sweeps the objective

range, moved at every repeated rounding in place of the random flips of the from-scratch pump of Section 7, with no target and a feasible point within the time limit as the endpoint. On the 272 0–1 instances of Section 5 that this pump handles and whose first LP takes at most 5 s, it solves 147 against 158 for the pump with random flips; run before it for part of the budget it adds 14 instances and loses 7, without significance. Code, logs and the full table are in the supplementary material. Future research should measure what the two parts are worth inside a full branch-and-cut, and should search for a good cutoff level.

What this does not show. Everything above is measured with the pump as a standalone heuristic. Inside a full branch-and-cut the pump is called once at the root, against a battery of cheaper heuristics and a bound that node processing improves anyway. As a consequence, the ceiling on what a modification of the pump can contribute there is set by how often it runs, not by how well. Our results say what the cutoff and the recovery do to the pump; they do not say that a solver will run faster, and we have not run the embedded experiment. E1 is the subset the plain pump can already handle, hence the ground most favorable to it, and its instances are smaller than the collection they come from (a median of about 72,000 nonzeros of the original model against 110,000 over all 463). Every claim of Section 6 is a claim about the subset. The five seeds enter the analysis only through the median, whose spread within an instance is large (Table 5: `none` finds a solution on every instance but on 856 of 880 runs). The design of Section 5 compares recoveries at fixed cutoff levels and does not search for a good level. The completion is measured as a count of LPs and by what it does to the time to the first solution and the number of rounds, not by wall-clock per call. The objective weighting inside the pump is at one setting, namely, SCIP’s default, which Section 2 argues should not matter in the pure-projection limit but which was not varied, its threshold not measured. MIPLIB 2017 with mostly binary variables is one collection of instances, on which the mechanism is strongest on covering-like structure. There the right reference is the Lagrangian heuristic of [10] rather than the pump, and nothing here is measured against it. E3 tunes one width per policy and per target on 20 instances, where a difference of two or three pairs in a hundred is noise; it reports TEST at $a = 0.5$ at a width the rule did not pick. Its sample is the third of MIPLIB 2017 on which a pure 0–1 pump with no help finds a first solution within 150 s, so its instances are the easier ones, with a median of 5,800 binaries against 1,700 and 15,000 at the quartiles. Recall that the target of (5) is defined from the best value known to the library, which the policies never see but which fixes how deep the target is, and the two values of a bracket what a user’s own choice of U can look like. Finally, the times in E3 are relative costs under a common load, compared only between policies that both succeed on an instance, and thus cannot be read as wall-clocks per call.

Reproducibility. Each table carries in the source the exact command that regenerates it from the raw result files; the aggregators use only the Python standard library and take the result file as an argument. We list what accompanies the paper in the declarations after the references.

A Tables by variant

The four tables of this appendix report the variants of Sections 6 and 7 one by one, the per-run counts behind their paired comparisons, and, for the target experiment, the paired comparisons behind Section 7; the text refers to them where their columns are read.

variant	with a solution		medians				completion LPs		
	inst.	runs	gap (%)	t_{first} (s)	rounds	LP iter.	calls	acc.	infeas.
none ₋	176/176	856/880	59.12	4.07	8 032	2 030	–	–	–
none _{0.5}	173/176	840/880	39.97	4.61	977	104 926	–	–	–
check ₋	176/176	856/880	58.87	3.99	7 695	2 124	–	–	–
check _{0.5}	173/176	840/880	30.82	4.12	834	105 064	–	–	–
check _{0.75}	173/176	838/880	29.69	4.04	836	96 620	–	–	–
check _{0.9}	173/176	840/880	27.96	4.11	873	86 760	–	–	–
check _{0.95}	173/176	840/880	29.39	4.09	944	80 909	–	–	–
FGL ₋	176/176	856/880	55.81	4.03	6 720	2 124	8 413 518	242	50 873
FGL _{0.5}	173/176	840/880	30.41	4.15	780	107 402	5 316 978	19 865	3 072 971
FGL _{0.95}	173/176	840/880	29.39	4.17	820	80 352	5 559 625	6 599	432 887
check _{0.5} ^{fb}	118/176	533/880	51.48	4.81	248	107 649	–	–	–
none _{0.5} ^{fb}	105/176	466/880	95.84	6.22	232	108 320	–	–	–

Table 5: E1, the 176 instances of 424 on which the pilot finds a solution, 5 seeds each. The median gap is the median over instances of the per-instance median over seeds; t_{first} , rounds and LP iterations are medians over all runs of the variant, those without a solution included (over the runs with one for t_{first}). *inst.* counts the instances on which at least one of the five seeds finds a solution, *runs* the runs that do. The last three columns count, for the completion variants, the completion LPs solved (*calls*), those whose solution was accepted, and those found infeasible.

variant	with a solution		medians				completion LPs		
	inst.	runs	gap (%)	t_{first} (s)	rounds	LP iter.	calls	acc.	infeas.
none ₋	27/248	79/1 240	100.00	19.83	354	28 622	–	–	–
none _{0.5}	23/248	70/1 240	100.00	19.75	261	32 250	–	–	–
check ₋	26/248	78/1 240	100.00	19.52	325	27 574	–	–	–
check _{0.5}	23/248	70/1 240	100.00	19.87	244	29 582	–	–	–
check _{0.75}	22/248	68/1 240	100.00	19.86	249	29 374	–	–	–
check _{0.9}	23/248	72/1 240	100.00	19.60	262	28 958	–	–	–
check _{0.95}	23/248	69/1 240	100.00	19.57	246	29 278	–	–	–
FGL ₋	26/248	78/1 240	100.00	19.82	317	27 494	3 192 751	28	3 170 429
FGL _{0.5}	23/248	69/1 240	100.00	19.82	242	29 368	3 187 843	53	3 187 787
FGL _{0.95}	23/248	69/1 240	100.00	19.73	245	28 800	3 196 657	57	3 190 506
check _{0.5} ^{fb}	11/248	27/1 240	100.00	15.40	64	44 872	–	–	–
none _{0.5} ^{fb}	9/248	24/1 240	100.00	11.11	66	47 584	–	–	–

Table 6: E2, the 248 instances of 424 on which the pilot finds nothing; columns as in Table 5. The median gap is 100 for every variant: each fails to find a solution, on its median seed, on more than half the instances.

comparison (A vs. B)	instances			p -value		median diff. $B - A$, non-tied
	A better	tie	A worse	sign test	Holm	
check ₋ vs. none ₋	1	246	1	1.0000	1.0000	+44.0
check _{0.5} vs. none _{0.5}	5	242	1	0.2188	1.0000	+10.7
none _{0.5} vs. none ₋	9	233	6	0.6072	1.0000	+1.5
check _{0.5} vs. check ₋	9	235	4	0.2668	1.0000	+1.8
check _{0.95} vs. check ₋	9	233	6	0.6072	1.0000	+0.1
check _{0.95} vs. check _{0.5}	3	239	6	0.5078	1.0000	-10.0
FGL _{0.5} vs. check _{0.5}	1	245	2	1.0000	1.0000	-5.1
FGL _{0.95} vs. check _{0.95}	2	246	0	0.5000	1.0000	+3.1
FGL ₋ vs. check ₋	0	248	0	1.0000	1.0000	-
FGL _{0.5} vs. check ₋	10	233	5	0.3018	1.0000	+1.2
check _{0.5} ^{fb} vs. check _{0.5}	3	235	10	0.0923	1.0000	-54.0
none _{0.5} ^{fb} vs. none _{0.5}	3	235	10	0.0923	1.0000	-50.8

Table 7: E2, the twelve declared comparisons paired per instance on the same endpoint as Table 3 (final gap, 100 for a run without a solution): instances on which the first variant is better, tied, worse; sign test on the non-tied; Holm correction over the twelve; and the median paired difference over the non-tied instances, which on this subset mostly reflects the jump from 100 to a finite gap.

comparison (A vs. B)	A only	B only	both	neither
<i>E1: the pilot finds a solution (176 instances)</i>				
check ₋ vs. none ₋	0	0	856	24
check _{0.5} vs. none _{0.5}	0	0	840	40
none _{0.5} vs. none ₋	8	24	832	16
check _{0.5} vs. check ₋	8	24	832	16
check _{0.95} vs. check ₋	8	24	832	16
check _{0.95} vs. check _{0.5}	0	0	840	40
FGL _{0.5} vs. check _{0.5}	0	0	840	40
FGL _{0.95} vs. check _{0.95}	0	0	840	40
FGL ₋ vs. check ₋	0	0	856	24
FGL _{0.5} vs. check ₋	8	24	832	16
check _{0.5} ^{fb} vs. check _{0.5}	6	313	527	34
none _{0.5} ^{fb} vs. none _{0.5}	6	380	460	34
<i>E2: the pilot finds nothing (248 instances)</i>				
check ₋ vs. none ₋	2	3	76	1 159
check _{0.5} vs. none _{0.5}	1	1	69	1 169
none _{0.5} vs. none ₋	17	26	53	1 144
check _{0.5} vs. check ₋	17	25	53	1 145
check _{0.95} vs. check ₋	16	25	53	1 146
check _{0.95} vs. check _{0.5}	1	2	68	1 169
FGL _{0.5} vs. check _{0.5}	0	1	69	1 170
FGL _{0.95} vs. check _{0.95}	2	2	67	1 169
FGL ₋ vs. check ₋	0	0	78	1 162
FGL _{0.5} vs. check ₋	16	25	53	1 146
check _{0.5} ^{fb} vs. check _{0.5}	12	55	15	1 158
none _{0.5} ^{fb} vs. none _{0.5}	9	55	15	1 161

Table 8: Found or not found, paired per run, for the twelve comparisons: runs on which only the first variant finds a solution, only the second, both, neither. E1 above, E2 below.

Table 9: Paired per-instance comparisons. W/T/L is the number of instances on which the first policy wins, ties, or loses against the second on the *outcome*, i.e. on the per-instance median success defined in Table 4; p is the exact two-sided sign test on the non-ties; p_{Holm} is the Holm correction over the 12 declared tests, namely 2 values of $a \times 2$ families (benchmark, non-benchmark) $\times 3$ comparisons — the *all* rows are reported for information only and are not part of the Holm family. *Faster* counts, among the instances solved by both policies, how many times the first policy’s median time to success is lower than the second’s by at least 5%, how many times the two tie within that tolerance, and how many times the second’s is lower.

Comparison	N	W/T/L	p	p_{Holm}	faster W/T/L
$a = 0.5$ — benchmark					
NAIVE vs. TEST	43	3/39/1	0.625	1.000	13/6/6
NAIVE vs. COMPLETION	43	4/35/4	1.000	1.000	12/4/8
TEST vs. COMPLETION	43	2/37/4	0.688	1.000	13/4/7
$a = 0.5$ — non-benchmark					
NAIVE vs. TEST	91	4/84/3	1.000	1.000	22/7/26
NAIVE vs. COMPLETION	91	5/83/3	0.727	1.000	33/5/16
TEST vs. COMPLETION	91	5/82/4	1.000	1.000	29/11/13
$a = 0.5$ — all					
NAIVE vs. TEST	134	7/123/4	0.549	—	35/13/32
NAIVE vs. COMPLETION	134	9/118/7	0.804	—	45/9/24
TEST vs. COMPLETION	134	7/119/8	1.000	—	42/15/20
$a = 0.9$ — benchmark					
NAIVE vs. TEST	43	1/41/1	1.000	1.000	13/8/10
NAIVE vs. COMPLETION	43	0/40/3	0.250	1.000	16/7/9
TEST vs. COMPLETION	43	0/40/3	0.250	1.000	16/10/6
$a = 0.9$ — non-benchmark					
NAIVE vs. TEST	91	3/86/2	1.000	1.000	28/16/25
NAIVE vs. COMPLETION	91	0/86/5	0.062	0.688	39/7/26
TEST vs. COMPLETION	91	0/85/6	0.031	0.375	45/19/7
$a = 0.9$ — all					
NAIVE vs. TEST	134	4/127/3	1.000	—	41/24/35
NAIVE vs. COMPLETION	134	0/126/8	0.008	—	55/14/35
TEST vs. COMPLETION	134	0/125/9	0.004	—	61/29/13

References

- [1] T. Achterberg, T. Berthold. Improving the feasibility pump. *Discrete Optimization* 4(1):77–86, 2007. <https://doi.org/10.1016/j.disopt.2006.10.004>
- [2] D. Baena, J. Castro. The Chebyshev center as an alternative to the analytic center in the feasibility pump. *Optimization Letters* 17(8):1757–1790, 2023. <https://doi.org/10.1007/s11590-023-02018-4>
- [3] L. Bertacco, M. Fischetti, A. Lodi. A feasibility pump heuristic for general mixed-integer problems. *Discrete Optimization* 4(1):63–76, 2007. <https://doi.org/10.1016/j.disopt.2006.10.001>
- [4] T. Berthold, A. Lodi, D. Salvagnin. Ten years of feasibility pump, and counting. *EURO Journal on Computational Optimization* 7(1):1–14, 2019. <https://doi.org/10.1007/s13675-018-0109-7>
- [5] T. Berthold, A. Lodi, D. Salvagnin. *Primal Heuristics in Integer Programming*. Cambridge University Press, 2025. <https://doi.org/10.1017/9781009574792>
- [6] N. Boland, A. Eberhard, F. Engineer, A. Tsoukalas. A new approach to the feasibility pump in mixed integer programming. *SIAM Journal on Optimization* 22(3):831–861, 2012. <https://doi.org/10.1137/110823596>
- [7] N. Boland, A. Eberhard, F. Engineer, M. Fischetti, M. Savelsbergh, A. Tsoukalas. Boosting the feasibility pump. *Mathematical Programming Computation* 6(3):255–279, 2014. <https://doi.org/10.1007/s12532-014-0068-9>
- [8] S. Bolusani, M. Besançon, K. Bestuzheva, A. Chmiela, J. Dionísio, T. Donkiewicz, J. van Doornmalen, L. Eifler, M. Ghannam, A. Gleixner, et al. The SCIP Optimization Suite 9.0. ZIB Report 24-02-29, Zuse Institute Berlin, 2024. arXiv:2402.17702.
- [9] M. Cacciola, A. Forel, A. Frangioni, A. Lodi. The differentiable feasibility pump. In *Integer Programming and Combinatorial Optimization (IPCO 2025)*, Lecture Notes in Computer Science 15620, pp. 157–171, Springer, 2025. https://doi.org/10.1007/978-3-031-93112-3_12
- [10] A. Caprara, M. Fischetti, P. Toth. A heuristic method for the set covering problem. *Operations Research* 47(5):730–743, 1999. <https://doi.org/10.1287/opre.47.5.730>
- [11] S. S. Dey, A. Iroume, M. Molinaro, D. Salvagnin. Improving the randomization step in feasibility pump. *SIAM Journal on Optimization* 28(1):355–378, 2018. <https://doi.org/10.1137/16M1095962>
- [12] M. Fischetti, F. Glover, A. Lodi. The feasibility pump. *Mathematical Programming* 104(1):91–104, 2005. <https://doi.org/10.1007/s10107-004-0570-3>
- [13] M. Fischetti, A. Lodi. Local branching. *Mathematical Programming* 98(1–3):23–47, 2003. <https://doi.org/10.1007/s10107-003-0395-5>
- [14] M. Fischetti, D. Salvagnin. Feasibility pump 2.0. *Mathematical Programming Computation* 1(2–3):201–222, 2009. <https://doi.org/10.1007/s12532-009-0007-3>
- [15] B. Geißler, A. Morsi, L. Schewe, M. Schmidt. Penalty alternating direction methods for mixed-integer optimization: a new view on feasibility pumps. *SIAM Journal on Optimization* 27(3):1611–1636, 2017. <https://doi.org/10.1137/16M1069687>
- [16] A. Gleixner, G. Hendel, G. Gamrath, T. Achterberg, M. Bastubbe, T. Berthold, P. Christophel, K. Jarck, T. Koch, J. Linderoth, M. Lübbecke, H. D. Mittelmann, D. Ozyurt, T. K. Ralphs, D. Salvagnin, Y. Shinano. MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation* 13(3):443–490, 2021. <https://doi.org/10.1007/s12532-020-00194-3>
- [17] S. Hanafi, J. Lazić, N. Mladenović. Variable neighbourhood pump heuristic for 0–1 mixed integer programming feasibility. *Electronic Notes in Discrete Mathematics* 36:759–766, 2010. <https://doi.org/10.1016/j.endm.2010.05.096>
- [18] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2):65–70, 1979.

- [19] J. Lazić, R. Todosijević, S. Hanafi, N. Mladenović. Variable and single neighbourhood diving for MIP feasibility. *Yugoslav Journal of Operations Research* 26(2):131–157, 2016. <https://doi.org/10.2298/YJOR140417027L>
- [20] G. Mexi, T. Berthold, D. Salvagnin. Using multiple reference vectors and objective scaling in the feasibility pump. *EURO Journal on Computational Optimization* 11:100066, 2023. <https://doi.org/10.1016/j.ejco.2023.100066>
- [21] `scipopt/scip`: Solving Constraint Integer Programs. Zuse Institute Berlin, commit `dba4b2a` (24 August 2026), with `scipopt/soplex` at commit `f0dbc81` (8 August 2026). <https://github.com/scipopt/scip>

Statements and Declarations

Funding. Work partially supported by MiUR, Italy.

Competing interests. The author declares no competing interests.

Data and code availability. The SCIP patches, job scripts, instance lists, raw SCIP logs, the solution files written by every run together with their independent validation on the original models, the from-scratch pumps of Sections 2 and 7 (the sweep trial of Section 8 included), the census of the rounded points behind Table 1 included, with their job scripts, instance lists and raw result files, and the scripts that regenerate every table and figure are provided in the repository <https://github.com/mfischetti-arch/fp-objective-cutoff>, at the tagged commit named in the submission, and will be archived with a persistent identifier upon acceptance.

Use of AI tools. Two AI assistants were used under the author’s direction: Claude (Anthropic) to draft and edit text, to write and debug the experimental scripts and to run internal adversarial reviews of the manuscript, and ChatGPT (OpenAI, model GPT-5.6) for an independent adversarial audit of the manuscript before submission. The author reviewed the text, the numbers and the code, and takes full responsibility for the content.